# 1. Problem Statement, Objectives, and Assumptions

## Problem Statement

Organising and visualising examination seating arrangements while accounting for reserved or blocked seats can be tedious to manage manually. An automated command-line tool is required to generate a grid-based seating layout based on user-defined room dimensions and specific reserved seat coordinates.

## Objectives

- Accept user input for the number of rows and the number of seats per row.
- 
- Allow users to specify reserved seats via a comma-separated format (e.g., 1-2,2-3) or indicate that there are none.
- 
- Parse and store reserved seat coordinates dynamically in a list.
- 
- Use nested loops to iterate through every row and column combination to display the complete seating grid.
- 
- Clearly distinguish reserved seats by displaying "R" and available seats by their coordinate identifiers (row-col).
- 

## Assumptions

- 

Room dimensions (rows and columns) are provided as valid positive integers.

- 
- 

Reserved seats are entered in the exact format row-col matching the generated coordinate system.

- 

## 2. Algorithm, Flowchart, and Pseudocode

### Algorithm

1. 

Prompt the user for the number of rows and seats per row (cols).

2. 
3. 

Prompt the user to enter reserved seats or "none".

4. 
5. 

Initialize an empty list called reserved_seats.

6. 
7. 

Check if the reserved input is neither "none" nor empty:

8. 

- 

Split the input string by commas and strip any surrounding whitespace to populate reserved_seats.

- 

9. 

Print a header indicating that "R" represents a reserved seat.

10. 
11. 

Start an outer loop for each row r from 1 to rows:

12. 

- 

Start an inner loop for each column c from 1 to cols.

- 

- Construct the seat string as r-c.
- 
- Check if seat exists in reserved_seats. If true, print "R" with a tab separator; otherwise, print the seat coordinate with a tab separator.
- 
- Move to the next line after completing each row.
- **Flowchart Representation**

**Pseudocode**

## 3. Python Source Code and Test Cases

### Python Source Code

```
        int input "Number of rows: "
       int input "Seats per row: "
            input "Reserved seats (example 1-2,2-3), or none: "

if                    "none" and                    ""
   for       in                ","


      "\nR = Reserved seat" for   in range 1          1
   for   in range 1        1
            str      "-"   str
      if      in
```

```
                "R"       "\t"
        else
                          "\t"
```

(Source code adapted from).

### Test Cases & Sample Output

| Test Case ID | Scenario Description | User Inputs | Expected Output / Behavior |
|---|---|---|---|
| TC-01 | Standard Seating Grid with No Reserves | Rows: 3 | |

Cols: 3

Reserved: none | Generates a 3x3 grid with coordinates from 1-1 to 3-3 with no reserved markings. | | TC-02 | Seating Grid with Specific Reservations | Rows: 2

Cols: 4

Reserved: 1-2, 2-4 | Generates a 2x4 grid where seats 1-2 and 2-4 display as R. |

**Simulated Terminal Output (for TC-02)**

## 4. Conclusion

The nested loop examination seating generator efficiently automates the layout design of exam halls or classrooms. By leveraging nested iteration combined with string parsing and conditional checks, it provides a flexible mechanism to map seating grids and handle reserved slots clearly.